COMPENG 4DV4

Very Large Scale Integration System Design

Mini Project 1 - Arithmetic Logic Unit

Qasim Saadat - saadatq - 400378059

February 11, 2026

ALU Operations Description

I will use a 2’s compliment signed decimal to display the inputs and outputs for a few operations for readability while the actual inputs and outputs have 7 bits of integer and 5 fraction bits.

Signed Addition

Signed addition occurs when the instruction code is 000.

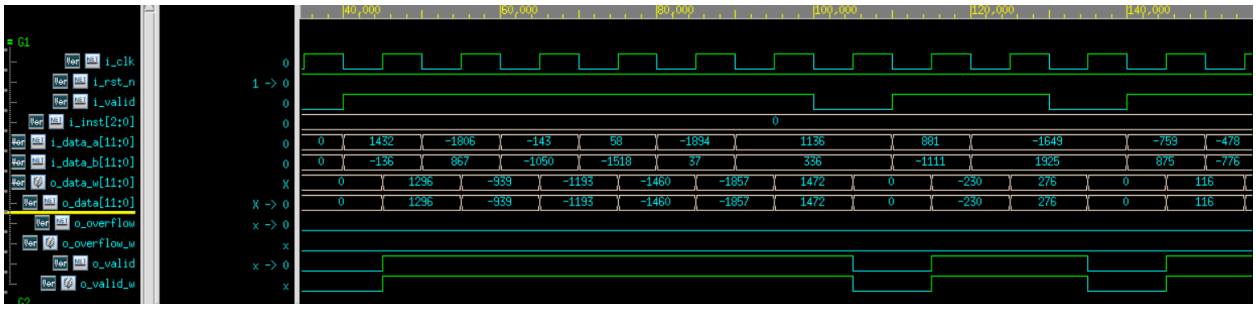
```
wire signed [12:0] signed_sum;

assign signed_sum = i_data_a + i_data_b;
```

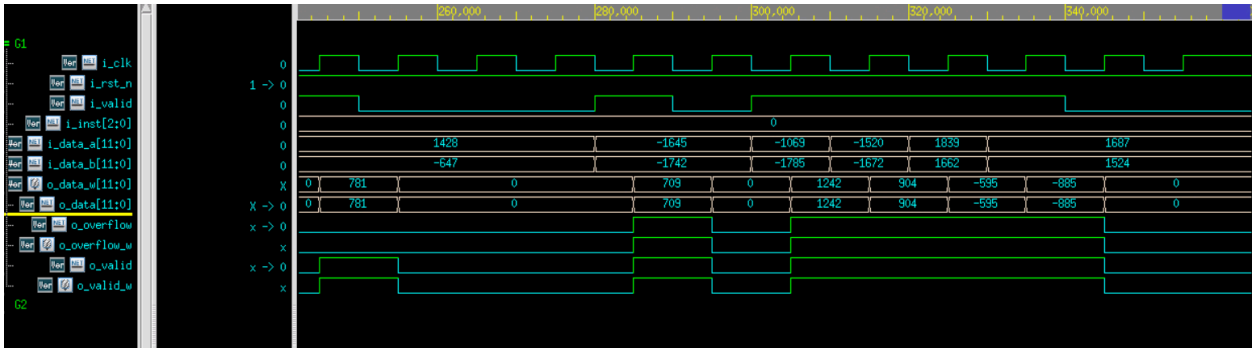
A wire called signed_sum was created and was assigned i_data_a + i_data_b. This wire was created with 13 bits rather than 12 as it is also used for instruction 110 and helps it out with that.

```
3'b000: begin
     o_data_w = signed_sum[11:0];                    // ADD
     o_overflow_w = (i_data_a[11] == i_data_b[11]) && (signed_sum[11] != i_data_a[11]);
end
```

For the overflow bit it occurs only when the most significant bit of the input data (the sign bit) are both the same while the output has a different value for that same bit. As the addition of the two bits with the same sign can cause an overflow and when this causes an overflow the summation would have a different sign than our inputs.



We can see from the output that our signed multiplication is working well from when neither, 1 input, or both inputs are negative. The o_valid is also working as expected.



The overflow bit is also working as intended. The range of our output bit is -2048 to 2047, and since -1069 + (-1742) will give us a sum that is outside of this range, there will be an overflow. Which does occur from the overflow bit going high.

Signed Subtraction

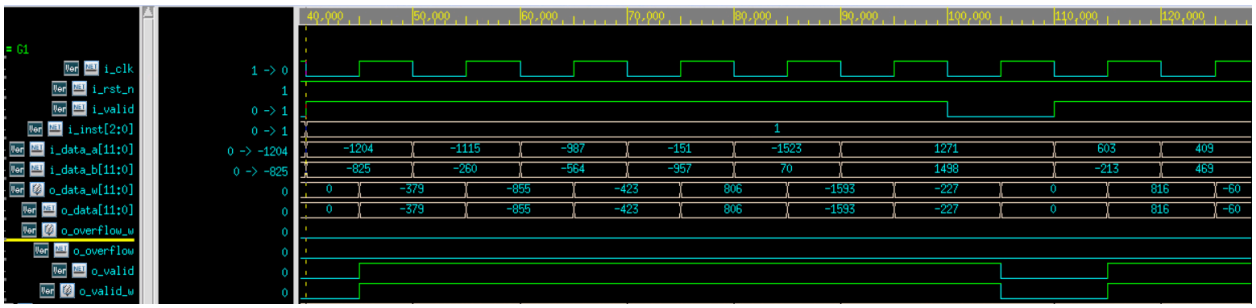
Signed Subtraction occurs when the instruction code is 001.

```
end
3'b001: begin
     o_data_w = i_data_a - i_data_b;                    // SUB
     o_overflow_w = (i_data_a[11] != i_data_b[11]) && (o_data_w[11] != i_data_a[11]);
end
```

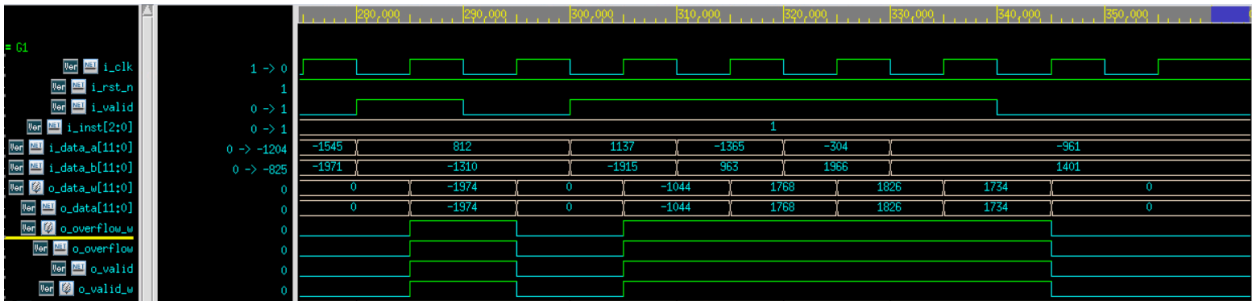
The inputs for i_data_a and i_data_b are signed so that we perform signed subtraction.

For the overflow bit we have two checks that must be true for their to be an overflow. Our first check is i_data_a[11] != i_data_b[11], which checks if the inputs have different signs since an overflow can only occur if they are different since that can cause the result to be above our bounds. The second check is if the output has a different sign bit than the first input since if the first check is true and the result has a different sign bit than the first input then the subtraction causes the result to go out of bounds causing the sign bit of the result to change. This would tell us that there is an overflow.

We can see from the results below that our subtraction is working as intended. The subtraction between the inputs -1204 and -825 is -379 which is what we can see from the results below from our waveform.



From our waveform below we can confirm that our overflow works as intended as a subtraction between 812 and -1310 results in a value of 2122 which will result in an overflow since our output has 12 bits. This is confirmed from our operation's waveform as the overflow bit is set to high.



Signed Multiplication

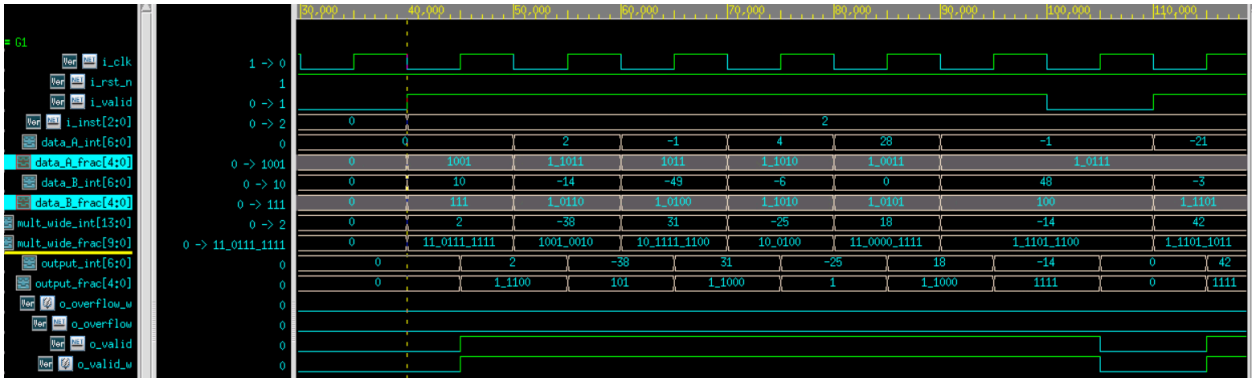
Signed Multiplication occurs when the instruction code is 010.

```
3'b010: begin
      o_data_w    = mult_fixed_point; // MULT
      o_overflow_w = (mult_wide[23:17] != {7{mult_wide[16]}});
end
```

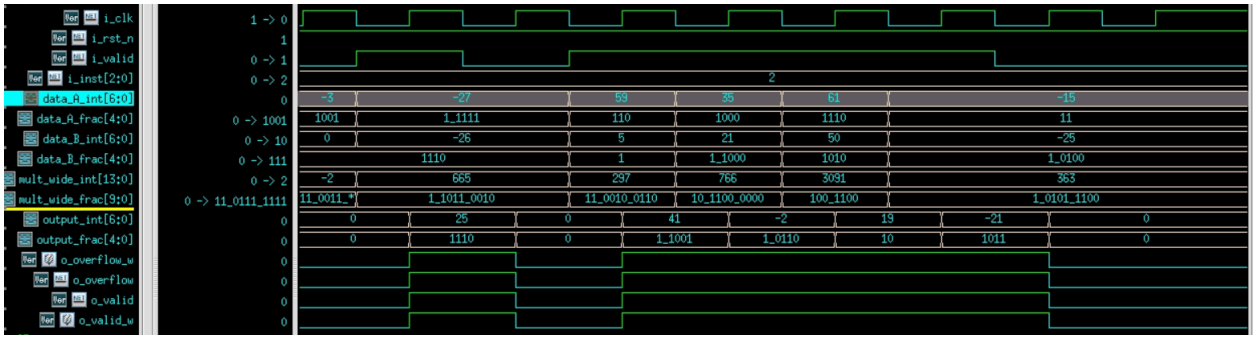
```
wire signed [23:0] mult_wide;
wire signed [11:0] mult_fixed_point;
assign mult_wide = i_data_a * i_data_b;
assign mult_fixed_point = (mult_wide + (1 <<< 4)) >>> 5;
```

Created two wires, one that is 12 bits and another that is 24 bits. The 24 bit is created to be able to store all the data from the multiplication to not lose any bits. In the 12 bit wire we right shift the whole multiplication by 5 since the input values have 7 bits of integers and 5 bits for fractions. We want to ensure we keep our output in the same notation and currently the 24 bit wire has 14 bits of integers and 10 bits for fractions. We also add a 1 <<< 4 for rounding.

For the overflow bit we check if all the bits from bit 17-23 are the same as bit 16. If they are the same then there is no overflow otherwise there is an overflow.



Our Multiplication works as intended, I separated the integers and fractions as readability. We have inputs of 0.28125 and 10.21875, This would give us an output of 2.874. This is what we received in our output (the integer we get is 2 and the fraction is 0.875). This confirms that our multiplication works as intended.



Since our range is -128.96875, 127.96875, without looking at the fraction bits the integer bits of -27 and -26 when multiplied results in a result larger than 127.96875 resulting in an overflow. We can see from the waveform above that during this operation, the overflow bit is set to high.

MAC

MAC occurs when the instruction code is 011.

```
wire signed [11:0] mult_fixed_point;
wire signed [12:0] MAC_summation;
assign mult_fixed_point = (mult_wide + (1 <<< 4)) >>> 5;
assign MAC_summation = mult_fixed_point + MAC_accumulate;
```

```
3'b011: begin
        o_data_w = MAC_summation[11:0]; // MAC
        MAC_accumulate = o_data_w;
        o_overflow_w = (mult_wide[23:17] != {7{mult_wide[16]}}) || (MAC_summation[12] != MAC_summation[11]);
end
```

We reuse the Mult_fixed_point wire for MAC but we also create another wire (MAC_summation_ which adds the accumulate to the mult result. We have the summation as 13 bits instead of 12 to avoid overflows. After setting the output data to MAC_summation, we also set the accumulate to the output data. This is so for the next set of values we can add the result of their multiplication to the previous MAC result.

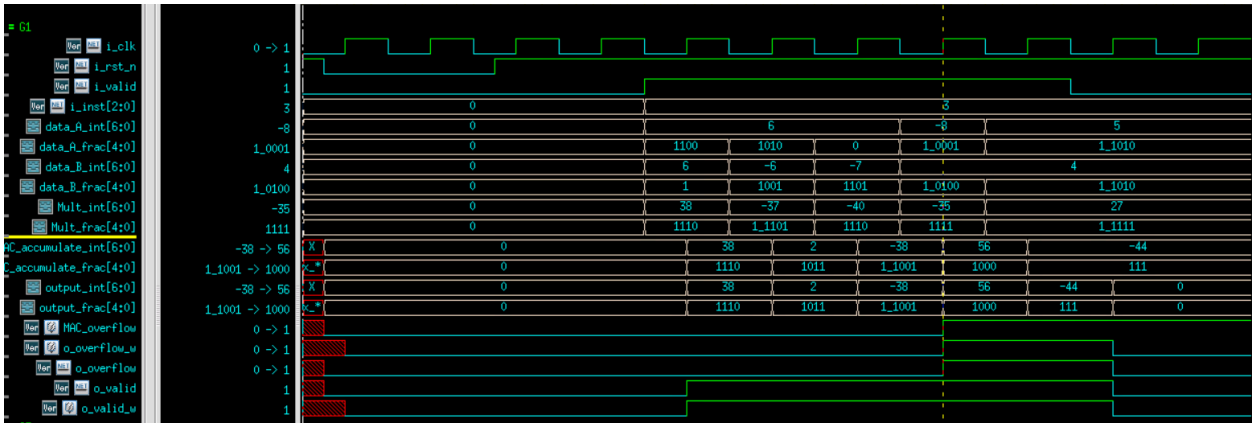
For the overflow bit we use the same overflow check as the multiplication, as the overflow can occur within the multiplication. We also have another check for the addition, since we have the addition as 13 bits we can check bit 13 and bit 12 and there is an overflow if they are different.

```
always@(posedge i_clk or negedge i_rst_n) begin
    if(!i_rst_n) begin
        o_data_r <= 0;
        o_overflow_r <= 0;
        o_valid_r <= 0;
        MAC_accumulate <= 0;
        MAC_overflow <= 0;
    end else begin
        o_data_r <= o_data_w;
        if (i_valid && i_inst == 3'b011) begin
                MAC_overflow <= o_overflow_w | MAC_overflow;
                o_overflow_r <= o_overflow_w | MAC_overflow;
        end else begin
                o_overflow_r <= o_overflow_w;
        end
        o_valid_r <= o_valid_w;

        if (i_valid && i_inst != 3'b011) begin
                MAC_accumulate <= 0;
                MAC_overflow <= 0;
        end

    end
end
```

We reset the MAC_overflow and MAC_accumulate once the instruction code is no longer for MAC. We keep the overflow bit set throughout the MAC once it has been set.



The accumulate is properly storing the output values and it adds that value to the new Mult value. We can see that our overflow bit remains high throughout the duration of the MAC.

XNOR

XNOR occurs when the instruction code is 100.

```
3'b100: o_data_w = ~(i_data_a ^ i_data_b);                    // XNOR
```

Performed bit wise operations for the XNOR operation as per the instructions. The XNOR gate functions where if both the inputs are the same then the output is high and if they are different then the output is low. We are checking this bitwise and this functionality is confirmed in the waveform screenshot below.

For example the xnor of 010100001000 and 111010000110 is 010001110001 which is what we see below as the first input and output from our waveform.

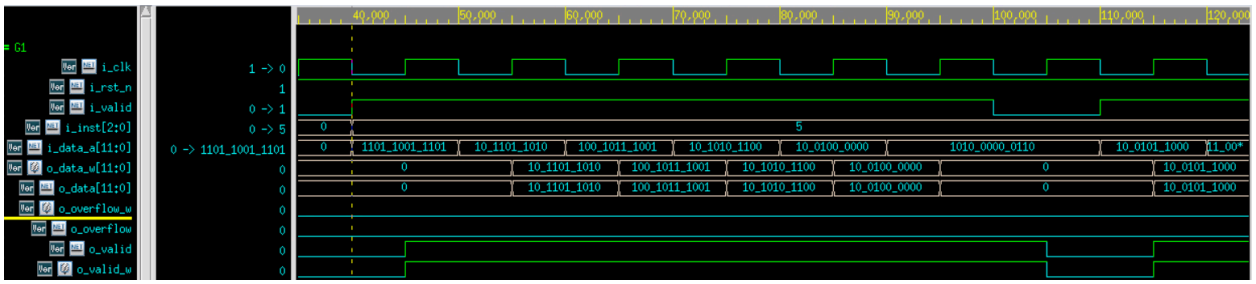
ReLU

ReLU occurs when the instruction code is 101.

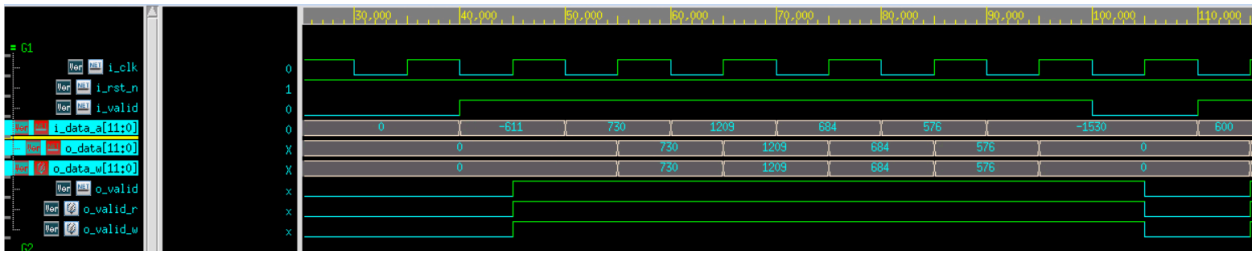
```
3'b101: o_data_w = (i_data_a[11] == 0) ? i_data_a : 12'd0; //ReLU
```

This operation checks if the most significant bit for i_data_a is 0 which indicates positive or above 0. If it is true output will return the value of i_data_a otherwise it will output 0. This functionality is confirmed in our ALU with the waveform screenshot below.

In the first input, since the sign bit is showing 1 the value of i_data_a is negative which means the output should be one. This is what we can see in the waveform below.



From the waveform below (by setting the notation to two's compliment), we can see that all the negative inputs return an output of 0 while the positive inputs return an output of the value of i_data_a.



Mean

Mean occurs when the instruction code is 110.

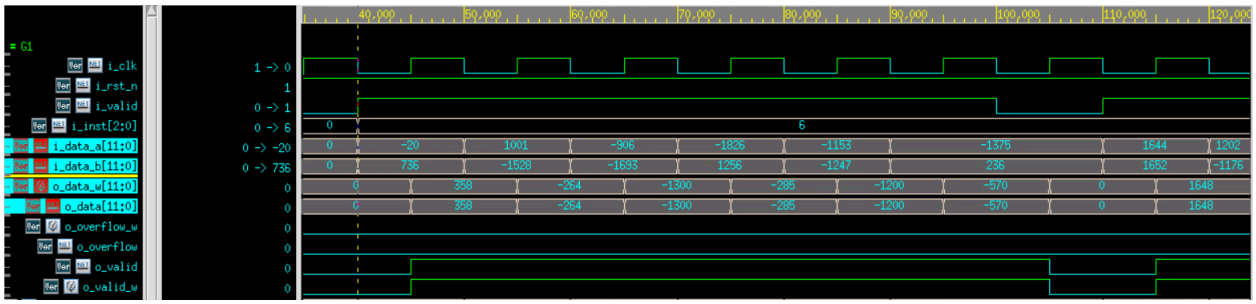
```
wire signed [12:0] signed_sum;

assign signed_sum = i_data_a + i_data_b;

3'b110: o_data_w = (signed_sum) >>> 1; //Mean
```

I used the 13 bits for signed_sum so when we perform a right shift there are no overflows. The right shift divides the value by a power of two since we are operating in base 2. This operation is performed in the waveform screenshot below.

For example: Our first inputs are -20 and 736. Which gives us a mean of (736-20)/2 = 358 and that is the result we obtain in our waveform.



Absolute Max

Absolute Max occurs when the instruction code is 111.

```
wire [11:0] abs_a, abs_b;
```

```
assign abs_a = i_data_a[11] ? (~i_data_a + 1'b1) : i_data_a;
assign abs_b = i_data_b[11] ? (~i_data_b + 1'b1) : i_data_b;
3'b111: o_data_w = (abs_a > abs_b) ? abs_a : abs_b; // Absolute Max
```

If the number inputted is negative for a or b, I use 2's compliment to convert it to a positive number. The output is the larger value between a and b.

In our waveform below we can see that our operation correctly converts negative numbers to positive and correctly determines the larger number between the two inputs.

In the first input and output, we have inputs of 2012 and 2037. The absolute Max operation returns 2037 which is the larger number.

For the second example, we have inputs of 15 and -40. Since we are looking for the absolute max, we find the absolute max for i_data_b which is a negative. This gives us the two numbers 15 and 40 where 40 is the larger number. This is the result I obtain in the waveform below.